Metodología Estructurada para el Desarrollo Asistido por IA

Esta metodología define un marco de trabajo riguroso diseñado para interactuar con Inteligencia Artificial en el desarrollo de software. El objetivo principal es evitar la entropía del código, las alucinaciones de la IA y el colapso del contexto, imponiendo restricciones, contratos y un flujo de trabajo inquebrantable.

A diferencia de las interacciones conversacionales libres, este enfoque trata a la IA como un sistema que requiere reglas explícitas, límites duros y una estructura de memoria organizada.

Pilar 1: Gestión de Contexto por Demanda (Contexto Navegable)

Uno de los mayores errores al trabajar con IA es proporcionarle demasiada información de golpe (archivos monolíticos). Esto satura la ventana de contexto, haciendo que la IA ignore instrucciones críticas o se vuelva ineficiente.

La Regla:

- **Índices ligeros:** La IA siempre debe leer un archivo de "índice" pequeño (ej. ~250 tokens) que le indique dónde buscar más información.
- **Carga bajo demanda:** Las secciones detalladas, reglas específicas o convenciones solo se proporcionan a la IA si un *trigger* o condición se cumple (ej. "Si vas a crear un componente, lee el archivo de convenciones de UI").

Pilar 2: Metadatos, Contratos y Límites Estrictos

Cada pieza de código o archivo dentro del ecosistema debe estar firmemente definida antes de que la IA pueda intervenir. Se utiliza un sistema de anotaciones obligatorio en la cabecera de cada archivo.

Estructura de Anotación (Ejemplo Conceptual):
`@metadata(tipo, contrato, limite_lineas, dependencias)`

1. **El Tipo (kind):** Define qué es el archivo (componente, servicio, hook, utilidad). Esto le dice a la IA qué convenciones aplicar.
2. **El Contrato (contract):** Define explícitamente las entradas, salidas y efectos secundarios esperados.
3. **Límites Duros (limit): Esto NO es una sugerencia, es una restricción.** Establecer un límite estricto de líneas obliga a la IA y al desarrollador a mantener la modularidad.
    - *Ejemplo de límites:* Hooks (80 líneas), Componentes (120 líneas), Páginas (200 líneas), Servicios (150 líneas).
    - *Consecuencia:* Si el límite se rompe, la tarea no puede darse por terminada y obliga a refactorizar.

Pilar 3: "Enforcement" (Cumplimiento Obligatorio)

Las reglas sin mecanismos de validación son solo aspiraciones. Para que la IA trabaje de manera confiable, el sistema debe bloquear cualquier desviación.

- **Validación Automática:** Se deben establecer "hooks" o scripts que revisen el código generado antes de guardarlo definitivamente en el control de versiones.
- **Prohibiciones Absolutas:** La IA tiene reglas inquebrantables, por ejemplo:
    - No usar tipado dinámico (any).
    - No saltarse los pasos de validación.
    - No inventar convenciones que no estén en la documentación.

Pilar 4: El Flujo de Trabajo Inquebrantable (Ciclo de Vida)

La IA nunca debe simplemente "escribir código". Para cada tarea, debe seguir una secuencia estricta y predecible:

1. **CONTEXTO:** Leer el estado actual, las reglas del proyecto y entender el entorno.
2. **ANÁLISIS:** Evaluar el estado de la tarea y qué piezas del sistema están involucradas.
3. **PLAN: Obligatorio.** Definir y registrar un plan de acción antes de escribir una sola línea de código.
4. **INVESTIGACIÓN:** Consultar la memoria semántica para ver patrones o guías existentes para este problema.
5. **ANÁLISIS DE IMPACTO:** Comprender qué partes del sistema actual podrían romperse con los cambios.
6. **MUTACIÓN:** Escribir o modificar el código de forma táctica, sin destruir el formato o la lógica existente.
7. **VALIDACIÓN:** Ejecutar pruebas y validar límites/reglas (prerrequisito para terminar).
8. **INTEGRACIÓN:** Actualizar la documentación si hubo cambios en la arquitectura.
9. **APRENDIZAJE:** Documentar errores cometidos para no repetirlos.
10. **CIERRE:** Registrar métricas y cerrar la sesión de trabajo.

Pilar 5: Memoria Semántica y Prevención de Errores

La IA no "recuerda" entre sesiones a menos que se le provea de una base de conocimiento persistente.

Registro de Errores (Learning Loop):
Cuando la IA o el desarrollador cometen un error o toman una mala decisión arquitectónica, esto no debe perderse. Se documenta estructuradamente: *Error, Entrada/Output Esperado, Corrección.* Esto se inyecta en el contexto futuro para que el sistema "aprenda".

Dominios de Conocimiento:
Se estructura la base de conocimientos por dominios específicos. La IA consulta estos documentos como "libros" cuando la tarea lo requiere, evitando alucinaciones.

Pilar 6: Tolerancia Cero a la Deuda Técnica (El Rol del Ingeniero)

El mayor riesgo del desarrollo asistido por IA es la pereza intelectual. Esta metodología exige un cambio de mentalidad radical:

- **Comprensión Obligatoria:** Si el alumno pide generar un sistema complejo, está éticamente obligado a entender cómo interactúan los subsistemas. Si no lo entiende, la IA mentirá o asumirá atajos.
- **Trazabilidad (El Ojo Crítico):** El alumno debe desarrollar la capacidad de hacer la traza manual del código generado. Debe auditar línea por línea, desarrollando un ojo clínico (PCA).
- **Deuda Técnica Cero:** Es inaceptable y anti-ingenieril acumular deuda técnica bajo la excusa de la inmediatez. Al usar documentos de requisitos profundos, se reduce los errores y reconstrucciones a cero. *Si una máquina va a teclear por ti, tu único trabajo es pensar y no tienes excusa para hacerlo mal.*

Resumen para su aplicación práctica

Para aplicar esta metodología, no se requiere de un software específico, sino de una **disciplina de trabajo interactiva**:

1. **Limita siempre** la cantidad de texto que le pasas a la IA para no saturar su ventana de contexto.
2. **Define contratos y límites estrictos** desde el primer momento (si un script pasa de X líneas, oblígala a dividirlo).
3. Exige que te entregue un **plan paso a paso** de forma estructurada antes de pedirle que escriba el código final.
4. **Documenta sus errores** de forma sistemática y dáselos como contexto al inicio del siguiente chat para retroalimentar el sistema.

"El control arquitectónico y la comprensión del modelo son los pilares fundamentales para liderar a la IA."